Лекция 10. Алгоритмы распределения задач и ресурсов

От венгерского алгоритма и аукционов к консенсусу и многоагентному поиску путей

90 минут · две части по 45

Курс «Управление распределёнными робототехническими системами»

План лекции

1. Постановка задачи распределения задач и критерии качества
2. Таксономия Джерки — Матарич: какие задачи решаются точно
3. Задача о назначениях и венгерский алгоритм
4. Рыночные методы: протокол контрактных сетей
5. Ставки, последовательные и комбинаторные аукционы
6. Консенсусный аукцион CBBA: торги без аукциониста
7. Пороговые модели и динамическое перераспределение
8. Многоагентный поиск путей: постановка и критерии
9. Приоритетное планирование и поиск на основе конфликтов
10. Пожизненная постановка: как на самом деле работает склад

Синие — первая часть · янтарные — вторая часть

Что нужно помнить с прошлых занятий

Сегодня эти шесть идей понадобятся постоянно, поэтому освежим их до начала новой темы.

Понятие (лекция N)	Что это простыми словами	Где пригодится сегодня
Централизованная архитектура (лекция 3)	Один узел-диспетчер собирает все данные и решает за всю группу	Венгерский алгоритм требует именно такого центра
Децентрализованная архитектура (лекция 3)	Каждый робот решает сам и общается только с соседями по сети связи	Аукционы и CBBA работают без единой точки отказа
Связный граф связи и его диаметр (лекция 4)	Диаметр — наибольшее число пересылок между двумя роботами в сети	От связности и диаметра зависит сходимость CBBA
Протокол консенсуса (лекция 4)	Роботы обмениваются оценками с соседями и сходятся к общему значению	CBBA консенсусом согласует список победителей торгов
Пороговая модель разделения труда (лекция 5)	Робот берётся за работу, когда стимул задачи превышает его личный порог	Распределение задач, которое работает вообще без связи
Аукцион как механизм (лекция 9)	Правила торгов, при которых выгода каждого ведёт к хорошему общему итогу	Основа всех рыночных методов первой части

Сокращения и термины лекции

Все сокращения, которые встретятся на слайдах сегодня, собраны в трёх таблицах; первая — о постановке задачи.

Обозначение	Расшифровка	Смысл в одной фразе
MRTA	Multi-Robot Task Allocation — распределение задач в группе роботов	Кому из роботов какую задачу поручить
ST и MT	Single-Task / Multi-Task — однозадачный и многозадачный робот	Сколько задач робот выполняет одновременно
SR и MR	Single-Robot / Multi-Robot — одноробotная и многороботная задача	Сколько роботов нужно одной задаче
IA и TA	Instantaneous / Time-extended Assignment — мгновенное и с горизонтом	Назначаем только текущие задачи или очередь на будущее
ST-SR-IA, ST-SR-TA, ST-MR-IA	Класс задачи: три буквы осей через дефис	ST-SR-IA: робот берёт одну задачу, ей нужен один робот, назначаем сейчас
NP-трудная задача	Задача, для которой не известен алгоритм с полиномиальным временем	Точное решение при росте размера быстро становится недостижимым
ЦЛП	Целочисленное линейное программирование	Оптимизация линейной функции по переменным, равным 0 или 1

Сокращения и термины лекции (продолжение)

Вторая таблица — сокращения из рыночных и консенсусных методов распределения задач.

Обозначение	Расшифровка	Смысл в одной фразе
CNP	Contract Net Protocol — протокол контрактных сетей	Объявление задачи, ставки, контракт и отчёт об исполнении
SSI	Sequential Single-Item auction — последовательный аукцион по одной задаче	Задачи торгуются по очереди, ставка — прирост маршрута
WDP	Winner Determination Problem — задача определения победителей	Выбор лучших непересекающихся пакетов в комбинаторном аукционе
CBBA	Consensus-Based Bundle Algorithm — пакетный алгоритм на основе консенсуса	Аукцион без аукциониста: победителей согласуют обменом с соседями
ε (эпсилон)	Минимальный шаг повышения цены в аукционе Бертсекаса	Чем меньше ε, тем точнее итог и тем дольше идут торги
VRP и mTSP	Vehicle Routing Problem / multiple Travelling Salesman Problem	Маршрутизация нескольких машин или роботов по набору точек
БПЛА	Беспилотный летательный аппарат	Типичный носитель CBBA: связь между дронами редкая и неустойчивая

Сокращения и термины лекции (окончание)

Третья таблица — обозначения из многоагентного поиска путей, которому посвящена вторая половина второй части.

Обозначение	Расшифровка	Смысл в одной фразе
MAPF	Multi-Agent Path Finding — многоагентный поиск путей	Пути для всех роботов сразу, чтобы никто ни с кем не столкнулся
A*	Алгоритм «А-звёздочка»	Поиск кратчайшего пути с подсказкой — оценкой расстояния до цели
SOC	Sum Of Costs — суммарная стоимость	Сумма времён прибытия всех роботов к своим целям
CBS	Conflict-Based Search — поиск на основе конфликтов	Ветвление по столкновениям; метод полный и оптимальный
ECBS	Enhanced CBS — ограниченно субоптимальный вариант CBS	Решение не хуже заданного множителя от оптимума, но быстрее
PIBT	Priority Inheritance with Backtracking — наследование приоритетов	Быстрое правило шага для тысяч роботов в плотном потоке
LaCAM	Lazy Constraints Addition search for MAPF — ленивое добавление ограничений	Полный и очень быстрый метод для десятков тысяч роботов

Кто что делает: распределение задач

Разберём, как поручить задачи роботам: от точного венгерского алгоритма до рыночных торгов.

“

Распределение задач похоже на торг: у каждого робота своя цена за работу, а система ищет назначение с наименьшей общей ценой.

Рамка всей первой части

1 Постановка задачи MRTA

Начнём с точной постановки: что дано, что ищем и чем хорошее назначение отличается от плохого.

– Задача MRTA (Multi-Robot Task Allocation — распределение задач в группе роботов) отвечает на вопрос «кто что делает», а не «как сделать».

– Даны n роботов и m задач; стоимость c_{ij} показывает, во что обойдётся роботу i задача j: время, энергия, путь или риск.

– Решение описывают переменные x_{ij}: $x_{ij} = 1$, если задача j поручена роботу i, и $x_{ij} = 0$ в противном случае.

– Ограничения задают, сколько задач может взять робот, сколько роботов нужно задаче и в какие сроки её надо выполнить.

Пример

На складе двадцать роботов и пятьдесят заказов, стоимость — время подъезда к стеллажу. Плохое назначение легко удваивает время выполнения всей партии.

1 Два критерия качества назначения

Чтобы сравнивать назначения между собой, нужен числовой критерий; в задаче распределения их обычно два.

$$J_1 = \sum_{i=1}^{n} \sum_{j=1}^{m} c_{ij}\, x_{ij} \to \min, \qquad J_2 = \max_{i} \sum_{j=1}^{m} c_{ij}\, x_{ij} \to \min$$

- Здесь c_{ij} — стоимость задачи j для робота i, x_{ij} — переменная назначения (1 или 0), n — число роботов, m — число задач.
- J_1 (критерий MinSum) — суммарные затраты всего парка; J_2 (критерий MinMax, или makespan) — затраты самого загруженного робота.
- Оптимумы по двум критериям обычно не совпадают: ради малой суммы всю работу можно свалить на одного «дешёвого» робота.
- Пример со склада: по J_1 мы экономим общий пробег парка, а по J_2 — время, к которому будет собрана последняя посылка партии.

Выбор критерия — решение о целях системы, а не математическая тонкость.

2 Таксономия Джерки — Матарич (2004)

Постановок MRTA много; классификация по трём осям сразу говорит, какой математический аппарат применим.

Ось	Варианты	Что это значит	Пример со склада
Робот	ST — одна задача за раз; MT — несколько сразу	Сколько задач робот ведёт одновременно	MT: робот везёт груз и попутно строит карту
Задача	SR — нужен один робот; MR — нужна коалиция	Сколько исполнителей требует задача	MR: тяжёлый поддон везут два робота вместе
Горизонт	IA — назначение сейчас; TA — очередь на будущее	Учитываем ли будущие задачи	TA: роботу заранее дают цепочку из пяти заказов

Класс записывают тремя буквами осей: ST-SR-IA — однозадачные роботы, однороботные задачи, мгновенное назначение. Всего классов восемь.

2 Какой класс решается точно

Таксономия полезна тем, что сразу показывает границу между точно решаемыми и трудными задачами.

– Лишь простейший класс ST-SR-IA сводится к классической задаче о назначениях и решается точно за время порядка n^3.

– Класс ST-SR-TA, где роботу назначают очередь задач, равносилен маршрутизации нескольких машин и является NP-трудным.

– Класс ST-MR-IA, где задаче нужна коалиция роботов, сводится к задаче о покрытии множеств и тоже NP-труден.

– Остальные пять классов также NP-трудны, поэтому для них на практике ищут хорошее приближённое решение, а не оптимум.

Ключевая идея

NP-трудность означает, что точный алгоритм с полиномиальным временем неизвестен: с ростом числа роботов время точного решения взлетает. Поэтому большинство методов этой лекции — приближённые.

3 Задача о назначениях

Простейший класс ST-SR-IA — это классическая задача о назначениях; посмотрим, почему её нельзя решать перебором.

– Даны n роботов, n задач и квадратная матрица стоимостей размера n × n; каждому роботу нужно дать ровно одну задачу.

– Из матрицы выбирают по одному элементу в каждой строке и каждом столбце так, чтобы сумма выбранных была минимальной.

– Полный перебор невозможен: вариантов n!, и при двадцати роботах их уже около $2{,}4 \cdot 10^{18}$.

– Если роботов и задач не поровну, добавляют фиктивные строки или столбцы: «фиктивный робот» означает, что задача не распределена.

Факт

Перебор двадцати факториала вариантов не закончится никогда. Венгерский алгоритм решает ту же задачу точно за время порядка n^3, то есть за тысячи, а не миллиарды миллиардов шагов.

3

Задача о назначениях как целочисленная программа

Запишем ту же задачу формулами: это объяснит, почему она, в отличие от соседних классов, решается точно.

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij}\, x_{ij}, \quad \sum_{j=1}^{n} x_{ij} = 1 \;\forall i, \quad \sum_{i=1}^{n} x_{ij} = 1 \;\forall j, \quad x_{ij} \in \{0, 1\}$$

- Здесь $x_{ij} = 1$, если робот i назначен на задачу j, и 0 иначе; c_{ij} — стоимость такой пары.
- Первое ограничение требует, чтобы каждый робот получил ровно одну задачу, второе — чтобы у каждой задачи был ровно один исполнитель.
- Матрица ограничений вполне унимодулярна (каждый её минор равен 0, 1 или −1), поэтому можно разрешить $0 \le x_{ij} \le 1$: оптимум всё равно целый.
- Значит, задачу точно решает обычное линейное программирование, а специализированный венгерский алгоритм — ещё быстрее.

Редкий случай: целочисленная задача решается точно за полиномиальное время.

3 На чём держится венгерский алгоритм

Прежде чем идти по шагам, поймём два факта, на которых основан алгоритм: без них шаги кажутся фокусом.

– Факт 1: вычитание числа из целой строки или столбца сдвигает стоимость всех назначений одинаково, и лучшее назначение не меняется.

– Факт 2: если в неотрицательной матрице есть n нулей, по одному в каждой строке и столбце, их назначение оптимально.

– Такие нули называют независимыми; алгоритм вычитаниями добивается, чтобы их стало ровно n.

– Теорема Кёнига: наибольшее число независимых нулей равно наименьшему числу линий (строк и столбцов), накрывающих все нули.

Что это даёт

Теорема Кёнига превращает проверку оптимальности в подсчёт линий: накрыли все нули n линиями — оптимальное назначение уже есть; хватило меньшего числа — матрицу нужно ещё преобразовать.

3 Венгерский алгоритм по шагам

Шаги алгоритма лишь повторяют вычитания из двух фактов, пока в матрице не найдутся n независимых нулей.

1 — Редукция строк

Из каждой строки вычитаем её минимум; в каждой строке появляется хотя бы один нуль.

2 — Редукция столбцов

Из каждого столбца вычитаем его минимум; теперь нули есть и в каждом столбце.

3 — Проверка

Накрываем все нули наименьшим числом линий. Если линий n, независимые нули есть и можно выбирать назначение.

4 — Сдвиг

Иначе берём наименьший непокрытый элемент k, вычитаем его из непокрытых, прибавляем к дважды покрытым и снова проверяем.

5 — Назначение

Выбираем n независимых нулей, начиная со строк с единственным нулём; их позиции и дают оптимальное назначение.

Алгоритм Куна (1955) гарантирует оптимальное назначение; современные реализации работают за время порядка n^3.

3 Пример: четыре робота и четыре заказа

Проверим алгоритм на матрице 4 × 4 из пособия, где стоимость — время в секундах до цели.

– Строки матрицы времён: (82, 83, 69, 92), (77, 37, 49, 92), (11, 69, 5, 86), (8, 9, 98, 23); строка — робот, столбец — заказ.

– После вычитания минимумов строк (69, 37, 5, 8) и столбцов (только 15 в четвёртом) все нули накрываются тремя линиями, а нужно четыре.

– Наименьший непокрытый элемент равен 6; после сдвига на 6 нули уже нельзя накрыть меньше чем четырьмя линиями, и оптимум найден.

– Итог: робот 1 берёт заказ 3, робот 2 — заказ 2, робот 3 — заказ 1, робот 4 — заказ 4; общее время 69 + 37 + 11 + 23 = 140 с.

Проверка ответа

Сумма всех вычтенных чисел равна 69 + 37 + 5 + 8 + 15 + 6 = 140, то есть стоимости найденного плана. Так и должно быть: в итоговой матрице выбранные клетки нулевые.

3 Масштаб задачи в индустрии

Почему бы не решать всё венгерским алгоритмом? Посмотрим на масштабы реальных складов.

1 млн+

мобильных роботов во всём парке Amazon Robotics

5000

роботов на одном складе у ведущих китайских производителей

секунды

отводятся на пересчёт назначения при новом заказе

– Сам венгерский алгоритм справляется с тысячами строк за доли секунды; трудность в другом — собрать в центре свежую матрицу всех стоимостей.

– Заказы на складе приходят непрерывно, поэтому качество системы меряют пропускной способностью — числом заказов в час.

3 Где венгерский алгоритм не подходит

Алгоритм точен и быстр, но его ограничения архитектурные, и именно они толкают к децентрализованным методам.

– Нужен центр: диспетчер должен собрать полную матрицу стоимостей, а значит, иметь канал связи с каждым роботом.

– Центр — единая точка отказа: при обрыве связи с сервером роботы остаются без назначений.

– Роботы движутся, заказы появляются и исчезают, поэтому стоимости устаревают и назначение приходится пересчитывать.

– Алгоритм решает только класс ST-SR-IA и не умеет планировать очередь задач на будущее.

Ключевая идея

Оптимальность — не единственный критерий. На складе метод, дающий 95 % от оптимума, но переживающий обрыв связи, часто предпочтительнее точного. А венгерский алгоритм остаётся эталоном для оценки приближённых методов.

? Как думаете?

На складе двадцать роботов, новые заказы приходят каждые несколько секунд, связь с сервером иногда пропадает. Каким способом распределять заказы между роботами?

- **A** Венгерским алгоритмом при каждом изменении
- **B** Аукционом с локальным пересчётом
- **C** Полным перебором ради гарантии оптимума
- **D** Случайно: всё равно придётся перепланировать

? Как думаете?

На складе двадцать роботов, новые заказы приходят каждые несколько секунд, связь с сервером иногда пропадает. Каким способом распределять заказы между роботами?

- **A** Венгерским алгоритмом при каждом изменении
- **B** Аукционом с локальным пересчётом
- **C** Полным перебором ради гарантии оптимума
- **D** Случайно: всё равно придётся перепланировать

ОТВЕТ

Аукционом с локальным пересчётом: он даёт план, близкий к оптимуму, обновляется по частям и не требует собирать всю матрицу в одном месте.

4 Рыночная метафора

Раз центр — слабое место, попробуем распределять задачи так, как это делает рынок, — через торги.

– Задачи играют роль товара: их выставляют на торги, а роботы выступают покупателями или подрядчиками.

– Ставка робота — его собственная оценка затрат на задачу с учётом положения, загрузки и заряда батареи.

– Задача достаётся тому, кто выполнит её дешевле, а полная матрица стоимостей при этом нигде не собирается.

– Отказавший робот просто перестаёт делать ставки, и его задачи достаются другим без остановки системы.

Связь с лекцией 9

На девятой лекции мы говорили о проектировании механизмов: правила игры подбирают так, чтобы выгода каждого вела к хорошему общему итогу. Аукцион распределения задач — именно такой механизм.

4 Распределение задачи через аукцион

Схема одного раунда торгов показывает, кто кому и что отправляет.



Задача объявляется всем участникам, роботы отвечают ставками, равными их предельным издержкам, а аукционист выбирает победителя и закрепляет за ним задачу. Роль аукциониста может исполнять один из роботов, поэтому выделенный сервер не нужен.

5 Протокол контрактных сетей (CNP)

Самый старый и до сих пор популярный рыночный механизм — протокол контрактных сетей Смита (1980).



CNP (Contract Net Protocol) прост и естественно поддерживает динамику, но жаден: каждый контракт выгоден сейчас, а не с учётом будущих задач.

5 Пример раунда CNP на складе

Разберём один раунд протокола с числами: три робота торгуются за доставку контейнера.

- Задача: отвезти контейнер из точки P в точку Q, путь от P до Q — 60 м, роботы едут со скоростью 1 м/с, критерий — минимальное время.
- Робот a_1 свободен и стоит в 40 м от P, его ставка 40 + 60 = 100 с; робот a_2 стоит в 10 м, но освободится через 70 с, его ставка 140 с.
- Робот a_3 свободен и стоит в 25 м от P, его ставка 25 + 60 = 85 с; она лучшая, и контракт достаётся ему.
- Если a_3 по дороге обнаружит, что маршрут непроходим, он вернёт задачу менеджеру, и раунд повторится с новыми ставками.

Ключевая идея

Робот a_2 ближе всех к грузу, но проигрывает, потому что его ставка учла занятость. Поэтому ставку считают по всей загрузке робота, а не только по расстоянию.

5 Ставка как предельные издержки

Пример показал, что всё решает правильно посчитанная ставка; посмотрим, как её считать в общем случае.

– Предельная, или маржинальная, ставка равна приросту стоимости маршрута робота, если вставить в него новую задачу.

– Она учитывает всё, что робот уже пообещал сделать: занятый робот сам называет высокую цену и отсеивается.

– Геометрия учитывается автоматически: задача рядом с уже взятыми дешевле, чем задача на другом конце склада.

– Отдельные правила «не перегружай робота» и «бери ближнее» писать не нужно: их заменяет одна формула ставки.

Ключевая идея

Правильно выбранная ставка делает всю работу: балансировка нагрузки и учёт географии получаются сами собой, без отдельных правил в коде.

5 Последовательный аукцион и его гарантия

Если торговать задачи по одной с маржинальными ставками, получится метод с редкой для децентрализации гарантией.

– SSI (Sequential Single-Item auction — последовательный аукцион по одной задаче) выставляет задачи на торги по очереди.

– В каждом раунде каждый робот ставит прирост длины своего маршрута, и задача уходит роботу с наименьшим приростом.

– Результат Лагудакиса, Кёнига и соавторов (2005): в маршрутной задаче с критерием MinSum итог не более чем вдвое хуже оптимума.

– Это редкий случай, когда децентрализованный метод имеет доказанную оценку качества, а не только удачные эксперименты.

Пример

Четыре робота объезжают двадцать точек склада. Если наилучший возможный суммарный пробег равен 1 км, последовательный аукцион гарантированно даст не больше 2 км.

5 Аукционный алгоритм Бертсекаса

Для класса ST-SR-IA есть аукцион, который приходит к оптимуму задачи о назначениях, — алгоритм Бертсекаса (1988).

$$j^* = \arg\max_j \left(v_{ij} - p_j\right), \qquad p_{j^*} \leftarrow p_{j^*} + \left(b_{ij^*} - b'_i\right) + \varepsilon$$

- v_{ij} — полезность задачи j для робота i, p_j — цена задачи j (вначале 0), $b_{ij} = v_{ij} - p_j$ — чистая выгода робота от задачи.
- Свободный робот берёт задачу j* с наибольшей выгодой и поднимает её цену на разницу со второй по выгоде задачей b'_i плюс шаг ε.
- Теорема: торги кончаются за конечное число шагов, а итог хуже оптимума не более чем на n·ε; при ε < 1/n и целых стоимостях он оптимален.
- Большой ε ускоряет торги, малый повышает точность, поэтому на практике торги повторяют с постепенно уменьшающимся ε.

Прежний владелец перебитой задачи становится свободным и снова вступает в торги.

5 Комбинаторные аукционы

Поштучные торги не видят выгоды от объединения задач; комбинаторный аукцион исправляет этот недостаток.

– Робот ставит не на отдельную задачу, а на пакет задач целиком, и цена пакета может быть меньше суммы цен его частей.

– Аукционист решает WDP (Winner Determination Problem — задачу определения победителей): ищет дешёвый набор непересекающихся пакетов.

– Плата высока: пакетов 2^m, а WDP NP-трудна, поэтому на практике разрешают ставки только на пары и тройки близких задач.

– Тот же механизм подходит для коалиций: пакетом становится группа роботов, претендующая на многороботную задачу.

Пример

Забрать два заказа с соседних стеллажей за один рейс дешевле, чем делать два рейса. Поштучный аукцион эту выгоду не увидит, а комбинаторный — увидит.

Конец первой части

Мы решили, кто какую задачу берёт: точно — венгерским алгоритмом, приближённо — аукционами. После перерыва уберём аукциониста и научим роботов разъезжаться без столкновений.

Без центра и без столкновений

В первой части мы назначали задачи через центр и через аукционы; теперь уберём аукциониста (CBBA, пороги) и разведём маршруты роботов (MAPF).

6 Зачем аукцион без аукциониста

Все аукционы первой части опирались на аукциониста; посмотрим, что делать, когда связи «все с одним» нет.

– Даже если роль аукциониста передаётся от робота к роботу, в момент торгов нужна надёжная связь всех участников с одним узлом.

– У группы БПЛА (беспилотных летательных аппаратов) или подводных аппаратов связь редкая, меняется и не имеет центра.

– Чой, Брюне и Хау (2009) предложили CBBA (Consensus-Based Bundle Algorithm — пакетный алгоритм на основе консенсуса).

– Ставки роботы делают сами, а список победителей согласуют протоколом консенсуса, общаясь только с соседями.

Связь с лекцией 4

На четвёртой лекции роботы консенсусом приходили к общему среднему значению. В CBBA консенсус достигается не о числах, а о том, кто выиграл каждую задачу.

6 Две фазы алгоритма CBBA

CBBA повторяет построение пакета и согласование с соседями, пока роботы не перестанут спорить о задачах.

1 — Пакет

Робот жадно добавляет в свой пакет задачу с наибольшим приростом полезности маршрута, если его ставка выше известной лучшей.

2 — Обмен

Робот отправляет соседям список предполагаемых победителей z_i, их ставки y_i и время, когда сведения получены.

3 — Решение

По таблице правил робот для каждой задачи выбирает: принять сведения соседа, оставить свои или сбросить запись.

4 — Освобождение

Перебитый по задаче робот отдаёт её и все задачи, добавленные после неё: их ставки считались с её учётом.

Последний шаг принципиален: без него ставки на поздние задачи опирались бы на маршрут, которого больше нет.

6 Что гарантирует CBBA

Алгоритм полностью децентрализован; важно понять, при каких условиях он всё-таки приходит к общему решению.

– Условия: граф связи связен, а польза от каждой следующей задачи в пакете не больше, чем от предыдущей (убывающий прирост).

– Утверждение: тогда CBBA за конечное число итераций даёт назначение без конфликтов, где у каждой задачи один робот.

– Итераций консенсуса не больше порядка $D \cdot L_t \cdot m$: D — диаметр графа связи, L_t — наибольшая длина пакета, m — число задач.

– Качество: суммарная полезность не меньше половины оптимальной, как у жадной максимизации функций с убывающей отдачей.

Что это даёт роботам

В домашнем задании курса роботы склада распределяют заказы через CBBA без сервера, и качество плана заведомо не хуже половины лучшего. Для групп БПЛА CBBA стал фактическим стандартом.

6 Централизованное назначение и рыночное

Сведём вместе всё, что мы узнали о двух подходах к назначению задач.

Централизованное (венгерский алгоритм)

– Оно гарантирует точный оптимум для класса ST-SR-IA.

– Оно требует собрать в центре полную матрицу стоимостей.

– Оно останавливается целиком при отказе центра или обрыве связи с ним.

– При каждом новом заказе оно пересчитывает назначение заново.

Рыночное (аукционы, CBBA)

– Оно даёт решение, близкое к оптимуму, иногда с доказанной оценкой.

– Оно обходится ставками, которые каждый робот считает по своим данным.

– Оно переживает отказ робота: его задачи просто переторговываются.

– Оно поглощает новые заказы локальными торгами без остановки системы.

7 Что показывает эксперимент

Какой подход выигрывает на практике? Эксперимент сравнивает рынок с пороговой моделью при росте команды.



Сравнение рыночного подхода (сплошная линия) и порогового (пунктир) в задаче обработки событий: число обработанных событий и пройденная дистанция в зависимости от размера команды. Рыночный подход выигрывает по качеству, но требует связи; пороговый устойчивее при её ухудшении.

7 Пороговые модели: распределение без сообщений

Пороговая модель из пятой лекции распределяет задачи вообще без связи; это резерв на случай её потери.

– У каждого робота есть свой порог θ — уровень стимула задачи, при котором он, скорее всего, за неё возьмётся.

– Стимул s робот измеряет сам, локально: например, длину очереди посылок у своей зоны склада.

– Вероятность взяться за задачу задаёт функция отклика $T(s) = s^n / (s^n + \theta^n)$; при $s = \theta$ она равна одной второй.

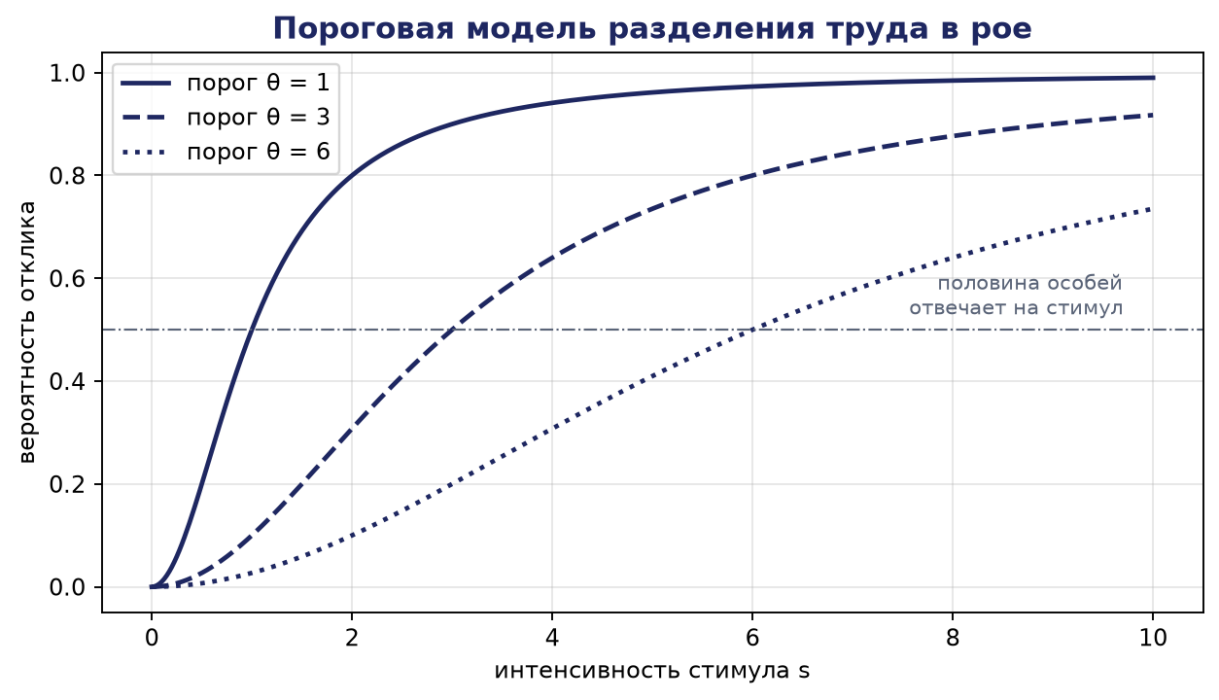
– Порог занятого робота понижается, а бездельничающего растёт, и со временем роботы сами становятся специалистами.

Кейс

У муравьёв разделение труда работает без единого сообщения: стимул растёт, пока задача не выполнена, и рано или поздно за неё берётся особь с подходящим порогом.

7 Пороговая функция отклика

Как параметры ϑ и n меняют поведение робота, видно на графиках функции отклика.



Слева — влияние порога θ при n = 10: чем ниже порог, тем раньше робот включается в работу. Справа — влияние крутизны n при θ = 50: при больших n отклик похож на резкий переключатель, при малых остаётся плавным и вероятностным.

7 Динамическое перераспределение задач

В реальной смене всё меняется ежеминутно, поэтому назначение приходится не строить один раз, а поддерживать.

– Появляются новые заказы, роботы ломаются и разряжаются, фактические стоимости отходят от расчётных, и вчерашний оптимум портится.

– Полный пересчёт при каждом событии дорог и вызывает «дребезг»: задачу перебрасывают между двумя почти равными роботами.

– Инкрементальная коррекция меняет только затронутую часть плана: новый заказ торгуется, задачи сломанного робота переторговываются.

– Против дребезга вводят гистерезис: задачу передают другому роботу, только если выигрыш больше заданного порога δ.

Пример

Робот с тремя заказами сломался. Его заказы возвращаются в общий пул, соседние роботы делают маржинальные ставки, и через раунд торгов заказы снова распределены, а остальной склад работает как работал.

8 Постановка задачи MAPF

Задачи распределены, но двадцать роботов, поехавших к стеллажам по кратчайшим путям, неизбежно столкнутся.

– MAPF (Multi-Agent Path Finding — многоагентный поиск путей) ищет пути сразу для k роботов на графе, обычно на сетке клеток склада.

– Время дискретно: за один такт робот переходит в соседнюю клетку или ждёт на месте; у каждого есть старт и цель.

– Вершинный конфликт возникает, когда двое роботов оказываются в одной клетке в один и тот же такт.

– Рёберный конфликт возникает, когда двое роботов меняются местами по одному ребру и сталкиваются посередине.

Ключевая идея

Задача похожа на школьную головоломку с фишками, но поиск оптимального решения NP-труден. При этом склады решают её для тысяч роботов каждую секунду.

8

Два критерия качества плана движения

Как и при назначении задач, план путей можно оценивать по сумме по всем роботам или по худшему роботу.

$$\mathrm{SOC} = \sum_{i=1}^{k} t_i, \qquad \mathrm{Makespan} = \max_i t_i$$

- t_i — такт, в который робот i окончательно прибыл в свою цель, k — число роботов.
- SOC (Sum Of Costs — суммарная стоимость) отражает среднюю эффективность парка; makespan — момент финиша последнего робота.
- Оптимум по одному критерию обычно не оптимален по другому: ускорив последнего робота, можно задержать девять остальных.
- На складе обычно важнее SOC, он отражает пропускную способность; при эвакуации или спасении важнее makespan.

Это те же MinSum и MinMax из начала лекции, только для путей. Важно и свойство полноты: находит ли метод решение, если оно существует.

9 Приоритетное планирование и резервирование клеток

Самый простой способ развести роботов — планировать их по очереди, резервируя уже занятые клетки.

– Роботов упорядочивают по приоритету и планируют по одному: каждый объезжает уже построенные пути старших.

– Путь ищут алгоритмом A* в пространстве «клетка плюс время», а занятые старшими пары «клетка, такт» резервируются.

– Метод очень быстрый, но неполный: он может не найти решение, даже когда оно существует.

– Так разводят роботов в домашнем задании курса: CBBA распределяет заказы, а резервирование клеток разводит маршруты.

Пример

В узком коридоре робот с низким приоритетом иногда не может проехать вовсе, хотя решение есть: достаточно было пропустить его первым.

9 Поиск на основе конфликтов (CBS)

Чтобы получить гарантию оптимума и не планировать всех роботов сразу, используют поиск на основе конфликтов.



CBS (Conflict-Based Search) полон и оптимален: если решение существует, он найдёт лучшее, но время растёт с числом конфликтов.

9 Семейство алгоритмов MAPF

Между приоритетным планированием и точным CBS лежит целое семейство методов разного масштаба.

Алгоритм	Свойство	Масштаб	Когда применять
A* в общем пространстве	Оптимален, но пространство растёт экспоненциально	До 3–5 роботов	Учебные примеры
Приоритетное планирование	Неполон, но очень быстр	Сотни	Когда важна скорость
CBS	Полон и оптимален	Десятки	Когда нужна гарантия оптимума
ECBS	Не хуже заданного множителя от оптимума	Сотни	Практический компромисс
PIBT	Быстрое правило шага с наследованием приоритетов	Тысячи	Плотные потоки роботов
LaCAM	Полон и очень быстр	Десятки тысяч	Современный стандарт

Ограниченно субоптимальный алгоритм вроде ECBS даёт решение не хуже заданного множителя от оптимума, и часто это лучший практический выбор.

Инженерный выбор

На складе триста роботов, и заказы приходят непрерывно в течение всей смены. Каким способом разумнее планировать движение роботов?

A. CBS ради оптимальности каждого плана

B. Ограниченно субоптимальным алгоритмом в скользящем окне

C. Один раз спланировать все маршруты на смену вперёд

D. Полным перебором ночью, когда склад пуст

Инженерный выбор

На складе триста роботов, и заказы приходят непрерывно в течение всей смены. Каким способом разумнее планировать движение роботов?

- **A** CBS ради оптимальности каждого плана
- **B** Ограниченно субоптимальным алгоритмом в скользящем окне
- **C** Один раз спланировать все маршруты на смену вперёд
- **D** Полным перебором ночью, когда склад пуст

ОТВЕТ

Ограниченно субоптимальным алгоритмом в скользящем окне: точный оптимум для трёхсот роботов быстро не считается, а поток заказов обесценивает дальний план.

10 Пожизненная постановка: как на самом деле работают склады

Квиз показал, что дальний план бесполезен; посмотрим, как ставят задачу, когда заказы не кончаются.

– В пожизненной постановке задачи не кончаются: робот довёз заказ и сразу получает следующий.

– Планировать маршруты до конца смены бессмысленно: через минуту появятся новые заказы, и всё изменится.

– Поэтому маршруты перепланируют в скользящем окне: строят план на несколько десятков тактов и регулярно обновляют его.

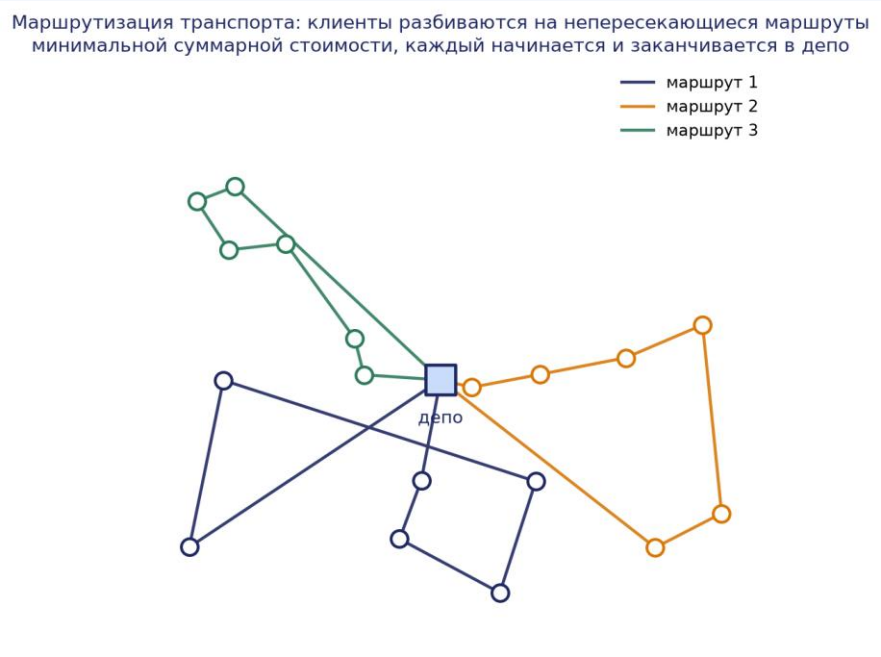
– Метрикой служит не длина плана, а пропускная способность — сколько заказов склад выполняет в час.

Кейс

На этой постановке проводится международное соревнование при поддержке Amazon Robotics: участники соревнуются в пропускной способности виртуального склада.

10 Маршрутизация как разбиение множества

Назначение и маршрутизация тесно связаны: классическая задача маршрутизации транспорта решает их одновременно.



Задача маршрутизации транспорта (VRP, Vehicle Routing Problem): из общего депо выходят несколько машин, каждая обслуживает свой непересекающийся набор клиентов. Нужно покрыть всех клиентов допустимыми по грузоподъёмности и времени маршрутами минимальной суммарной стоимости.

11

Разрыв между планом и исполнением

Даже идеальный план движения ломается в первую же минуту; разберёмся, почему и как этого избежать.

– План MAPF предполагает, что все роботы едут синхронно и точно по расписанию тактов.

– В реальности робот задерживается: буксует, объезжает человека, ждёт лифт или освобождения зарядной станции.

– Одна задержка каскадом ломает согласованность всего плана: роботы приходят в клетки не в те такты.

– Поэтому хранят порядок проезда роботов через общие клетки, а не точные моменты времени.

Ключевая идея

Хранить нужно не «робот А проедет перекрёсток на такте 17», а «робот А проедет перекрёсток раньше робота Б». Порядок переживает задержки, а расписание — нет.

11

Две половины складской задачи

Подведём итог: складская задача состоит из двух половин, и обе мы сегодня разобрали.

Распределение задач (MRTA)

– Оно отвечает на вопрос, какой робот едет к какому стеллажу.

– Его решают венгерским алгоритмом, аукционами или CBBA.

– Оно оптимизирует суммарные затраты или время завершения.

– Оно смотрит на текущие заказы или на очередь ближайших.

Поиск путей (MAPF)

– Он отвечает на вопрос, как всем разъехаться без столкновений.

– Его решают приоритетным планированием, CBS, ECBS, PIBT или LaCAM.

– Он оптимизирует пропускную способность склада.

– Он планирует в скользящем окне на несколько десятков тактов.

“

Назначить задачи и развести маршруты — это одна задача, разрезанная пополам ради вычислимости. Разрез удобен, но он же порождает заторы.

Главная нерешённая проблема области

12 Открытые проблемы

Закончим тем, что пока не решено: здесь есть место для курсовых и дипломных работ.

– Совместное решение назначения и маршрутизации: плохое назначение создаёт заторы, а заторы меняют реальную стоимость назначения.

– Учёт заряда батарей и очередей к зарядным станциям, которые сами становятся общим ресурсом парка.

– Гетерогенные парки, где роботы различаются скоростью, габаритами и грузоподъёмностью.

– Обучаемые методы пока уступают классическому поиску на тысячах роботов; лучше работают гибриды.

Факт

Попытки решать MAPF целиком обучением показали себя хуже классического поиска при тысячах роботов. Побеждают гибриды: обучение внутри поиска, а не вместо него.

Выводы по лекции

1. Таксономия MRTA делит задачи по трём осям; точно и быстро решается лишь класс ST-SR-IA.
2. Венгерский алгоритм находит оптимум за время порядка n^3, но требует центра с полной матрицей стоимостей.
3. Аукционы распределяют задачи ставками роботов, и маржинальная ставка сама учитывает загрузку и географию.
4. Последовательный аукцион хуже оптимума не более чем вдвое, а CBBA даёт не меньше половины оптимума.
5. Пороговые модели работают без связи, а гистерезис защищает перераспределение задач от дребезга.
6. MAPF разводит роботов: CBS полон и оптимален, а PIBT и LaCAM масштабируются до тысяч роботов.
7. Склад работает в пожизненной постановке: план обновляют в скользящем окне и хранят порядок проезда.

Вопросы для самопроверки

1. Чем различаются классы ST-SR-IA и ST-SR-TA и почему первый решается точно, а второй нет?

2. Почему вычитание минимума из строки не меняет оптимального назначения?

3. Что утверждает теорема Кёнига и как она проверяет оптимальность в венгерском алгоритме?

4. Как считается маржинальная ставка робота и почему она сама балансирует нагрузку?

5. Какую гарантию качества даёт последовательный аукцион и для какого критерия?

6. Почему в CBBA перебитый робот освобождает и все задачи, добавленные после потерянной?

7. При каких условиях CBBA сходится и какая доля оптимума при этом гарантирована?

8. Что такое дребезг назначений и как его подавляет гистерезис?

9. Чем вершинный конфликт отличается от рёберного? Почему приоритетное планирование неполно?

10. Почему на складе планируют в скользящем окне и хранят порядок проезда, а не моменты?